POLITÉCNICA

ETSI SISTEMAS INFORMÁTICOS

Examen de Fundamentos de Ingeniería del Software | **8 de junio de 2022**

Publicación de notas:	Lunes 20 de junio de 2022
Revisión:	22 de junio de 2022 de 10:30-12:30

Aula de Examen:_____
Nº asiento ocupado:_____
DNI con Letra:_____
Apellidos y Nombre: _____

FIRMA

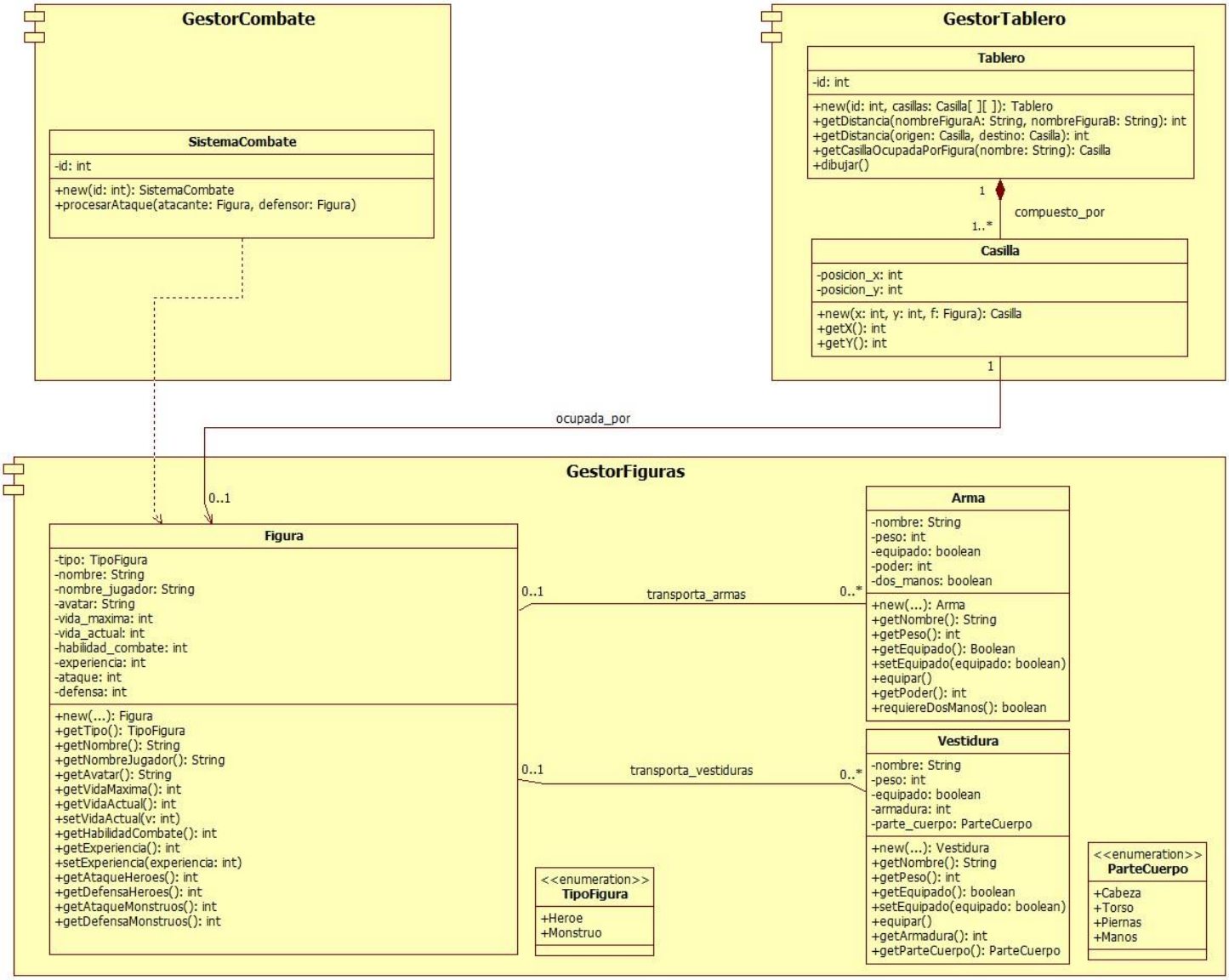
PROBLEMA 1 (3,5 puntos)

Una empresa especializada en el desarrollo de videojuegos ha iniciado un proyecto para desarrollar la versión digital de un juego de mesa de exploración de mazmorras. En este juego hay un tablero de dos dimensiones compuesto por casillas. Una casilla puede estar vacía u ocupada por una figura. Existen dos tipos de figuras: héroes y monstruos. Los héroes son figuras controladas por un jugador mientras que los monstruos son figuras controladas por una inteligencia artificial. Toda figura tiene un nombre único, un avatar para representarla gráficamente en el tablero, un atributo que indica los máximos puntos de vida que puede tener, un atributo que indica los puntos de vida que tiene actualmente y un atributo numérico que indica su nivel de habilidad de combate. Además, los héroes tienen puntos de experiencia y los monstruos tienen unos atributos 'ataque' y 'defensa' que representan, respectivamente, el poder de los ataques que pueden realizar y su defensa frente a los ataques de los que pueden ser objetivo. Los héroes (y solo los héroes) pueden transportar y equiparse armas y vestiduras. Todas las armas y vestiduras tienen un nombre único y un cierto peso. Además, toda arma tiene un atributo numérico 'poder' que representa el poder de los ataques que pueden realizar los héroes con ella y toda vestidura tiene un atributo numérico 'armadura' que representa la defensa aportada al héroe cuando éste es objetivo de un ataque. Para las armas también se debe conocer la cantidad de manos necesarias para equipárselas mientras que para las vestiduras se debe conocer la parte del cuerpo (Cabeza, Torso, Piernas o Manos) en la que se pueden equipar. Un héroe debe tener suficientes manos libres para equiparse un arma y para equiparse una vestidura no debe tener otra equipada en la misma parte del cuerpo. Un héroe puede transportar un objeto (arma o vestidura) sin necesidad de equiparse dicho objeto por lo que, de entre todos los objetos transportados por un héroe, algunos pueden estar equipados y otros no.

En la siguiente página se muestra un fragmento del **diagrama de clases de diseño** que ha realizado el equipo de desarrollo de software de la empresa para modelar el juego descrito anteriormente en el cual se han representado únicamente las clases correspondientes a los siguientes tres componentes del sistema: "GestorTablero", "GestorFiguras" y "GestorCombate". A continuación, se muestra la implementación del método `procesarAtaque(atacante: Figura, defensor: Figura)` de la clase SistemaCombate:

```
public void procesarAtaque(Figura atacante, Figura defensor) {
  int ataque, defensa;
  if(atacante.getTipo() == Figura.TipoFigura.Heroe) {
    ataque = atacante.getAtaqueHeroes();
  } else {
    ataque = atacante.getAtaqueMonstruos();
  }
  if(defensor.getTipo() == Figura.TipoFigura.Heroe) {
    defensa = defensor.getDefensaHeroes();
  } else {
    defensa = defensor.getDefensaMonstruos();
  }
  int factorAleatorio = (int) Math.round(((Math.random()-0.5)*4));
  int diffHabilidad = atacante.getHabilidadCombate()-defensor.getHabilidadCombate();
  int heridas = Math.max(0,ataque-defensa+diffHabilidad+factorAleatorio);
  defensor.setVidaActual(Math.max(0, defensor.getVidaActual()-heridas));
}
```

La implementación de los métodos `getAtaqueHeroes()` y `getAtaqueMonstruos()` es diferente, ya que el ataque de los héroes depende de las armas que éstos tengan equipadas y el ataque de los monstruos depende únicamente de su atributo 'ataque'. De manera similar, la implementación de los métodos `getDefensaHeroes()` y `getDefensaMonstruos()` es diferente, ya que la defensa de los héroes depende de las vestiduras que éstos tengan equipadas y la defensa de los monstruos depende únicamente de su atributo 'defensa'.



Apartado 1.1 (3 puntos)

a) Mejore el **diagrama de clases de diseño** proporcionado en el enunciado aplicando los **principios de diseño** que estime oportunos. Además de la información proporcionada por el enunciado, tenga en cuenta las siguientes indicaciones:

- Se busca reducir el acoplamiento entre los tres componentes, pero no el acoplamiento entre las clases pertenecientes a un mismo componente.
- El componente GestorCombate tiene que poder ser capaz de procesar ataques entre figuras de cualquier tipo sin necesidad de conocer los diferentes tipos de figuras existentes.
- El método `dibujar()` de GestorTablero muestra a los jugadores, por medio de una interfaz gráfica, el tablero del juego. En este tablero las figuras se representan mediante su avatar.

b) Identifique los **principios de diseño aplicados** en el apartado anterior e indique de manera clara y concisa, para cada uno de ellos: los **nombres de los elementos del diagrama de clases de diseño** que se han modificado o añadido resultado de la aplicación del principio y qué **beneficio** se ha obtenido con dicha aplicación.

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

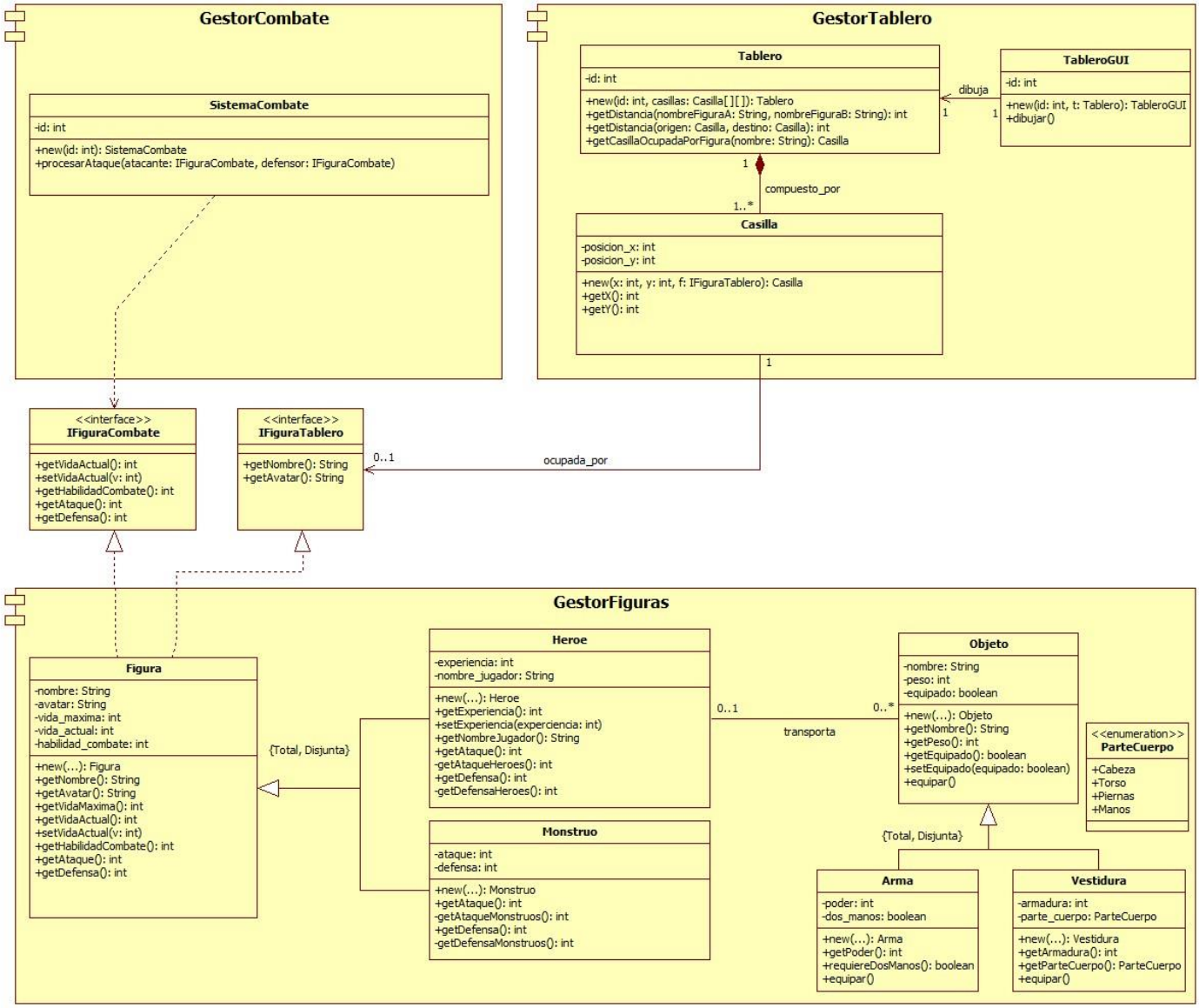
Principio de diseño aplicado:
Elementos modificados o añadidos:
Beneficio:

Apartado 1.2 (0,5 puntos)

Realice el **diagrama de componentes** del sistema en base exclusivamente al diagrama de clases de diseño que ha elaborado como solución del apartado 1.1.

Solución

Apartado 1.1



Con este diseño la implementación del método `procesarAtaque` de la clase SistemaCombate quedaría de la siguiente manera:

```
public void procesarAtaque(IFiguraCombate atacante, IFiguraCombate defensor) {
  int factorAleatorio = (int) Math.round(((Math.random()-0.5)*4));
  int diffHabilidad = atacante.getHabilidadCombate()-defensor.getHabilidadCombate();
  int heridas = Math.max(0,atacante.getAtaque()-defensor.getDefensa()+diffHabilidad+factorAleatorio);
  defensor.setVidaActual(Math.max(0, defensor.getVidaActual()-heridas));
}
```

Principio de diseño aplicado: Abstracción por especialización.
Elementos modificados o añadidos: Clases Figura, Heroe y Monstruo.
Beneficio: Se evita incluir en Figura atributos y métodos específicos de héroes y monstruos, es decir, atributos y métodos que no son necesarios para todos los tipos de figuras.

Principio de diseño aplicado: Abstracción por generalización.
Elementos modificados o añadidos: Clases Objeto, Arma y Vestidura.
Beneficio: Los atributos y métodos comunes de Arma y Vestidura se definen en Objeto evitando duplicidades innecesarias.

Principio de diseño aplicado: Principio SOLID de sustitución de Liskov.
Elementos modificados o añadidos: Clases SistemaCombate, Figura, Heroe y Monstruo.
Beneficio: El método procesarAtaque de la clase SistemaCombate no tiene que conocer detalles de las clases hijas de Figura. Este hecho posibilita que el componente GestorCombate sea capaz de procesar ataques entre figuras de cualquier tipo sin necesidad de conocer los diferentes tipos de figuras existentes.

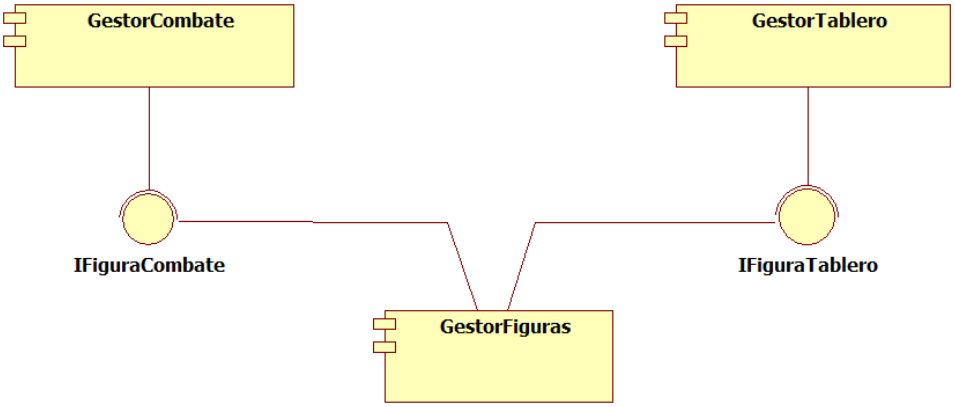
Principio de diseño aplicado: Principio SOLID de responsabilidad única.
Elementos modificados o añadidos: Clases Tablero y TableroGUI.
Beneficio: La clase Tablero pasa a tener una única responsabilidad ya que la responsabilidad de mostrar el tablero en la interfaz gráfica se ha trasladado a la clase TableroGUI. El principal beneficio es que un cambio en la forma en la que se dibuja el tablero no afectará a ninguna clase salvo a TableroGUI.

Principio de diseño aplicado: Principio SOLID de inversión de dependencias.
Elementos modificados o añadidos: Interfaces IFiguraCombate, IFiguraTablero. Clases Figura, Casilla, SistemaCombate.
Beneficio: Las clases SistemaCombate y Casilla ya no dependen de la implementación interna de la clase Figura sino de sus interfaces. De este modo, se reduce notablemente el acoplamiento ya que ningún componente depende de la implementación interna de otro componente.

Principio de diseño aplicado: Principio SOLID de segregación de interfaces.
Elementos modificados o añadidos: Interfaces IFiguraCombate, IFiguraTablero. Clase Figura.
Beneficio: La clase Figura ofrece dos interfaces, una específica para el componente GestorCombate y otra específica para el componente GestorTablero. De este modo, ninguna clase se ve forzada a depender de una interfaz que tenga métodos que no use.

Principio de diseño aplicado: Principio SOLID de abierto/cerrado.
Elementos modificados o añadidos: Interfaces IFiguraCombate e IFiguraTablero. Clases Figura, Heroe, Monstruo, Casilla y SistemaCombate.
Beneficio: Con la solución actual se posibilita el sustituir cualquiera de los componentes por otro que requiera y proporcione las mismas interfaces sin que el resto de componentes se vean afectados. Esto permite que se pueda extender el comportamiento del sistema sin modificar el código fuente de estos componentes.

Apartado 1.2


GestorCombate
GestorTablero
IFiguraCombate
IFiguraTablero
GestorFiguras